

Diseño conceptual y lógico de la base de datos

Sadane Geronimo Miguel Santiago Acevedo Virgues

Julian Camilo Corredor Rojas

Brayan Estif Calderon Gomez

Yerlinson Maturana Serna



Docente

Miguel Alfonso Varela

Universidad de la sabana

Diseño y Optimización de Bases de Datos

2026

Contenido

Análisis de requisitos	3
Contexto de Ecommify	3
Requisitos Funcionales	3
Requisitos No Funcionales	4
Identificación de Entidades del Dominio	4
Tabla de Volúmenes de Datos Esperados	5
Diseño Conceptual	6
Diagrama Entidad-Relación (ER)	6
Descripción de Entidades y Relaciones	6
Restricciones de Negocio	7
Diseño conceptual.....	8
Diseño lógico- modulo PostgreSQL	8
Diseño lógico preliminar- modulo MongoDB	12
Decisiones arquitectónicas justificadas	15
Matriz de decisión por entidad	15
Análisis del Teorema CAP aplicado	16
Trade-offs y mitigaciones.....	16
Anexos.....	17
1. Diccionario de datos resumido	17
2. Scripts SQL preliminares.....	20
3. Índices preliminares	24
4. Ejemplos de datos	25
5. Ejemplo de documento MongoDB complementario	29
6. Observación técnica.....	30

Análisis de requisitos

Contexto de Ecommify

Ecommify es una plataforma de comercio electrónico multivendedor especializada en productos tecnológicos. Permite que múltiples vendedores independientes ofrezcan sus productos a clientes en distintas regiones geográficas. El sistema centraliza la gestión de pedidos, pagos, envíos y reseñas, y requiere manejar tanto datos transaccionales de alta consistencia como datos analíticos de alto volumen y flexibilidad.

El dataset base proviene de Olist, un e-commerce real brasileño, con registros de ~99,000 clientes, ~99,000 pedidos, ~113,000 ítems de pedido, ~33,000 productos, ~3,000 vendedores y más de 1,000,000 registros de geolocalización.

Requisitos Funcionales

ID	Requisito
RF-01	El sistema debe registrar clientes con su información geográfica (ciudad, estado, código postal).
RF-02	El sistema debe gestionar pedidos con sus estados (pendiente, aprobado, en camino, entregado, cancelado).
RF-03	Cada pedido puede contener múltiples ítems, cada uno asociado a un producto y un vendedor.
RF-04	El sistema debe registrar los pagos de cada pedido (tipo, cuotas, monto). Un pedido puede tener múltiples pagos secuenciales.
RF-05	Los clientes deben poder dejar reseñas sobre sus pedidos (puntaje 1-5, comentario, fecha).
RF-06	El catálogo de productos debe incluir nombre, categoría, dimensiones y fotos.
RF-07	Los vendedores deben estar asociados a su ubicación geográfica.
RF-08	El sistema debe soportar traducción de categorías de producto (portugués → inglés).
RF-09	Deben poder consultarse los datos de geolocalización a partir del código postal.

Requisitos No Funcionales

ID	Requisito
RNF-01	Consistencia: Las transacciones de pedidos y pagos deben garantizar integridad ACID (PostgreSQL).

RNF-02	Escalabilidad: El catálogo de productos y las reseñas deben escalar horizontalmente (MongoDB).
RNF-03	Disponibilidad: El sistema debe estar disponible al menos el 99.5% del tiempo.
RNF-04	Rendimiento: Las consultas de pedidos por cliente deben responder en menos de 500ms.
RNF-05	Flexibilidad: El esquema de productos debe permitir atributos variables según categoría sin alterar el esquema base.
RNF-06	Seguridad: Los datos personales de clientes deben estar protegidos mediante control de acceso por roles.
RNF-07	Mantenibilidad: El esquema debe estar normalizado en mínimo 3FN para los datos transaccionales.

Identificación de Entidades del Dominio

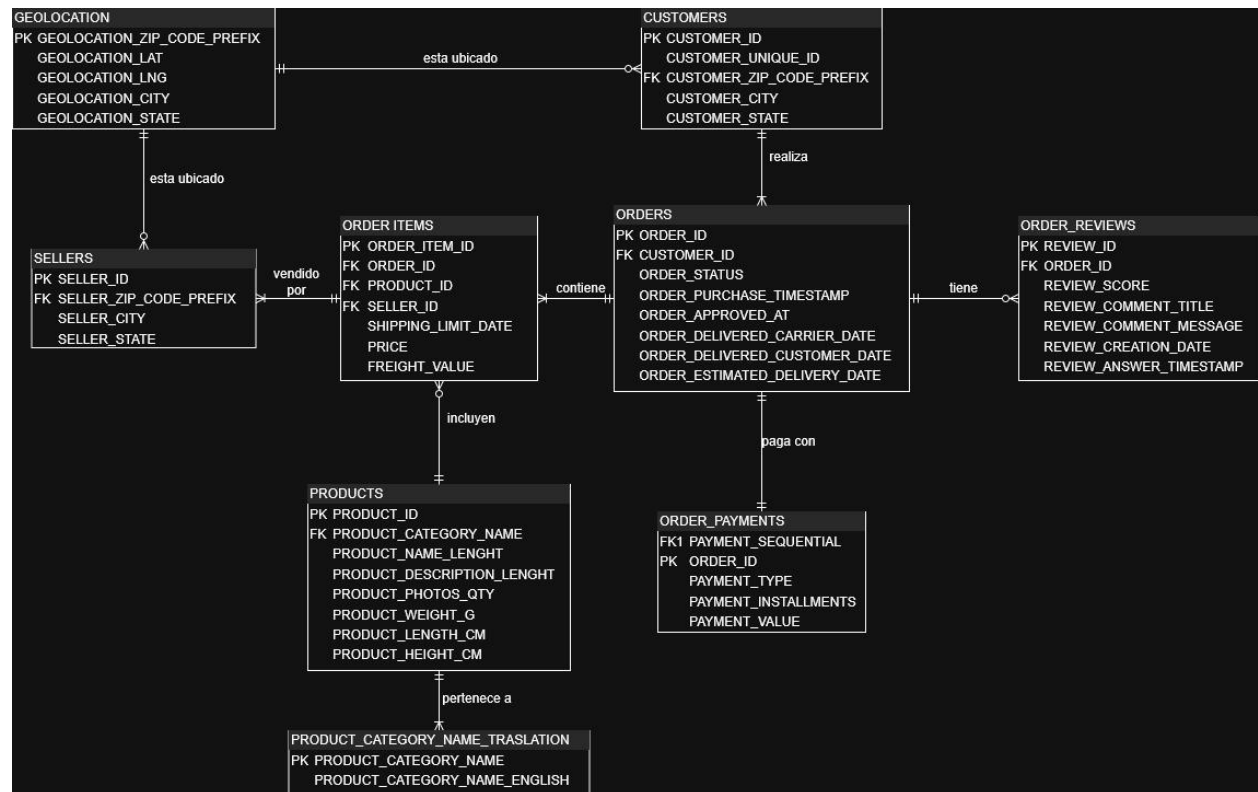
Entidad	Descripción	Fuente CSV
Customer	Cliente que realiza compras en la plataforma	olist_customers_dataset.csv
Order	Pedido realizado por un cliente	olist_orders_dataset.csv
OrderItem	Ítem individual dentro de un pedido	olist_order_items_dataset.csv
Product	Producto ofrecido en la plataforma	olist_products_dataset.csv
Seller	Vendedor que ofrece productos	olist_sellers_dataset.csv
Payment	Pago asociado a un pedido	olist_order_payments_dataset.csv
Review	Reseña de un cliente sobre un pedido	olist_order_reviews_dataset.csv
ProductCategory	Categoría de producto con traducción al inglés	product_category_name_translation.csv
Geolocation	Información geográfica asociada a un código postal	olist_geolocation_dataset.csv

Tabla de Volúmenes de Datos Esperados

Entidad	Registros actuales	Crecimiento estimado anual
Customer	~99,441	+30%
Order	~99,441	+30%
OrderItem	~112,650	+35%
Payment	~103,886	+30%
Review	~104,719	+40%
Product	~32,951	+20%
Seller	~3,095	+10%
ProductCategory	~70	Bajo
Geolocation	~1,000,163	+5%

Diseño Conceptual

Diagrama Entidad-Relación (ER)



Descripción de Entidades y Relaciones

Entidades

- **Geolocation:** Tabla de geolocalización asociada a un prefijo de código postal (geolocation_zip_code_prefix). Incluye latitud, longitud, ciudad y estado.
- **Customer:** Representa al comprador. Tiene un identificador único de cliente (customer_id) y uno único de persona (customer_unique_id). Se vincula a una geolocalización mediante customer_zip_code_prefix.
- **Seller:** Vendedor registrado en la plataforma. Se localiza geográficamente por seller_zip_code_prefix.
- **Order:** Pedido realizado por un cliente. Contiene el estado del pedido (order_status) y fechas del ciclo de vida: compra, aprobación, entrega al transportista, entrega al cliente y fecha estimada de entrega.
- **OrderItem:** Ítem dentro de un pedido. Es la entidad intermedia que relaciona pedidos, productos y vendedores. Incluye shipping_limit_date, price y freight_value.
- **Product:** Producto del catálogo. Incluye atributos de catálogo y físicos como nombre, descripción, cantidad de fotos, peso y dimensiones.
- **ProductCategoryNameTranslation:** Tabla de traducción de categorías de producto. Relaciona el nombre de la categoría en portugués con su traducción al inglés.
- **OrderPayment:** Pago asociado a un pedido. Un pedido puede tener múltiples pagos. Registra tipo de pago, número de cuotas y valor.
- **OrderReview:** Reseña del cliente sobre un pedido. Incluye puntaje, título del comentario, comentario, fecha de creación y fecha de respuesta.

Relaciones

Relación	Cardinalidad	Descripción
Geolocation → Customer	1:N	Un código geográfico puede estar asociado a muchos clientes
Geolocation → Seller	1:N	Un código geográfico puede estar asociado a muchos vendedores
Customer → Order	1:N	Un cliente puede realizar muchos pedidos
Order → OrderItem	1:N	Un pedido contiene uno o más ítems
Order → OrderPayment	1:1	Un pedido puede tener un pago

Order → OrderReview	1:0..N	Un pedido puede tener cero o varias reseñas
Product → OrderItem	1:0..N	Un producto puede tener 0 o muchos items
OrderItem → Seller	N:1	Muchos ítems pueden ser vendidos por el mismo vendedor
Product → ProductCategoryNameTranslation	1:N	Un producto puede tener varias categorías traducidas

Restricciones de Negocio

ID	Restricción
RN-01	Cada Order debe estar asociado a exactamente un Customer.
RN-02	Cada Order debe contener al menos un OrderItem.
RN-03	Cada OrderItem debe referenciar un Product existente.
RN-04	Cada OrderItem debe referenciar un Seller existente.
RN-05	El order_status solo puede tomar valores válidos del conjunto definido por el negocio.
RN-06	El OrderPayment.payment_value debe ser mayor a 0.
RN-07	El OrderItem.price debe ser mayor o igual a 0.
RN-08	El OrderItem.freight_value debe ser mayor o igual a 0.
RN-09	El OrderReview.review_score debe estar entre 1 y 5, inclusive.
RN-10	Un Order puede tener cero o varias OrderReview, pero cada reseña debe pertenecer a una sola orden.
RN-11	Un Customer debe estar asociado a un geolocation_zip_code_prefix existente.
RN-12	Un Seller debe estar asociado a un seller_zip_code_prefix existente.
RN-13	Un Product debe pertenecer a una categoría existente en ProductCategoryNameTranslation.
RN-14	Un mismo customer_unique_id puede estar asociado a múltiples customer_id si el cliente realiza compras en distintas sesiones.
RN-15	Un mismo geolocation_zip_code_prefix puede tener múltiples registros de latitud y longitud.

Diseño conceptual

Diseño lógico- modulo PostgreSQL

Para estructurar el núcleo transaccional de Ecommify, se **ha diseñado** un esquema relacional fuertemente tipado y normalizado hasta la Tercera Forma Normal (3FN). El objetivo principal aquí fue garantizar la integridad referencial y el cumplimiento de las propiedades ACID para las operaciones críticas de negocio: gestión de pedidos, pagos y control de inventario.

Esquema relacional normalizado (3FN)

A continuación, se detalla la estructura de las tablas principales que componen este módulo transaccional.

- **1FN — Atomicidad:** todos los atributos del esquema son escalares o tipos avanzados nativos de PostgreSQL (JSONB, arrays, composite, range). Los tipos avanzados no violan la 1FN porque el motor los trata como valores atómicos con operadores y rutas indexables.
- **2FN — Dependencias completas:** se eliminan las dependencias parciales sobre claves compuestas. Ejemplo: `order\_items` (PK compuesta `order\_id`, `order\_item\_id`) no contiene atributos que dependan solo de `order\_id`; los datos del pedido viven en `orders`.
- **3FN — Sin transitividad:** se eliminan dependencias transitivas. Ejemplo: `customer\_city` y `customer\_state` dependen funcionalmente de `customer\_zip\_code\_prefix`, por lo que se desnormalizan a la tabla maestra `geolocation\_zip`, evitando inconsistencias entre las ~1M filas de geolocation y los 99k clientes.
- **3FN como techo, no piso:** no se busca BCNF/4FN porque la sobre-normalización degrada el rendimiento OLTP (más JOINS por transacción). Se aplica desnormalización selectiva controlada en lecturas analíticas mediante vistas materializadas (no en tablas base).

Tabla: customers

Almacena la información transaccional clave de los compradores.

Columna	Tipo de Dato	Claves / Constraints	Descripción
customer_id	UUID	PK	Identificador único del cliente.
customer_zip_code	VARCHAR(10)	NN	Código postal para logística.
location	GEOMETRY(Point, 4326)	NN	Coordenadas exactas para cálculo de envíos.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de registro.

Tabla: sellers

Almacena la información transaccional clave de los vendedores.

Columna	Tipo de Dato	Claves / Constraints	Descripción
seller_id	VARCHAR(50)	PK	Identificador único del vendedor.
zip_code	VARCHAR(10)	NN	Código postal de origen.
location	GEOMETRY(Point, 4326)	NN	Coordenadas exactas para logística.

Tabla: orders

El corazón del sistema transaccional. Está aislada de los detalles del producto para mantener la 3FN.

Columna	Tipo de Dato	Claves / Constraints	Descripción
order_id	UUID	PK	Identificador único del pedido.
customer_id	UUID	FK (customers), NN	Cliente que realiza el pedido.
order_status	VARCHAR(20)	NN, CHECK status	Estado (creado, pagado, enviado).
created_at	TIMESTAMPTZ	NN	Fecha de creación. Servirá como clave de partición.
delivery_window	TSTZRANGE	NN	Rango de tiempo estimado para la entrega.

Tabla: order_items

Resuelve la relación muchos a muchos entre pedidos y el catálogo (cuyos detalles residen en MongoDB).

Columna	Tipo de Dato	Claves / Constraints	Descripción
order_item_id	UUID	PK	Identificador único de la línea.
order_id	UUID	FK (orders), NN	Pedido al que pertenece.
product_id	VARCHAR(50)	NN	Referencia al catálogo en MongoDB.
price	NUMERIC(10,2)	NN, CHECK (price > 0)	Precio congelado al momento de compra.
freight_value	NUMERIC(10,2)	NN, CHECK (>= 0)	Costo de envío calculado.

Tabla: payments

Registra las transacciones financieras.

Columna	Tipo de Dato	Claves / Constraints	Descripción
payment_id	UUID	PK	Identificador del pago.
order_id	UUID	FK (orders), NN	Pedido asociado.
payment_type	VARCHAR(20)	NN	Método (tarjeta, boleto, voucher).
payment_value	NUMERIC(10,2)	NN, CHECK (> 0)	Monto total procesado.
gateway_metadata	JSONB		Datos variables de respuesta de la pasarela de pagos.

Tabla: inventory

Controla el stock en tiempo real para evitar sobreventas.

Columna	Tipo de Dato	Claves / Constraints	Descripción
inventory_id	UUID	PK	Identificador del registro.
product_id	VARCHAR(50)	NN	Referencia al producto.
warehouse_id	UUID	NN	Almacén físico.

available_qty	INTEGER	NN, CHECK (>= 0)	Cantidad disponible para la venta.
reserved_qty	INTEGER	NN, CHECK (>= 0)	Cantidad separada por pedidos en proceso.

Aplicación de tipos avanzados y extensiones

Para optimizar el almacenamiento sin romper la integridad relacional, se han implementado los siguientes tipos y extensiones nativas:

1. JSONB en payments.gateway_metadata: Las pasarelas de pago devuelven estructuras muy variables. Usar JSONB permite indexar estos datos sin crear múltiples columnas dispersas que estarían nulas la mayoría del tiempo.
2. TSTZRANGE en orders.delivery_window: En lugar de columnas separadas para inicio y fin, este tipo de rango permite validar superposiciones y asegurar que el inicio sea menor que el fin a nivel nativo.
3. Extensión PostGIS: Aplicada en los campos location de customers y sellers. Esto permite realizar consultas espaciales (ej. ST_Distance) para calcular dinámicamente los costos de envío basados en la distancia real.
4. Extensión pgcrypto: Utilizada para la generación nativa de claves primarias UUID (gen_random_uuid()), mejorando la seguridad al no revelar volúmenes transaccionales mediante IDs secuenciales.

Estrategia de particionamiento

Considerando el volumen del dataset, se ha estructurado un particionamiento declarativo para la tabla orders:

- **Método:** Particionamiento por rango (RANGE) basado en created_at.
- **Estructura:** Particiones mensuales (ej. orders_2026_01).
- **Justificación:** Separa los datos "calientes" (pedidos recientes) de los "fríos" (históricos). Las consultas operativas diarias leen solo la partición actual, mejorando drásticamente el rendimiento I/O.

Estrategia de Ingesta: Mapeo de los 9 Datasets de Olist

Para asegurar la trazabilidad y demostrar el manejo completo de las fuentes de datos, se ha diseñado la siguiente matriz que detalla la distribución de los 9 datasets originales hacia nuestra arquitectura híbrida:

Dataset Original de Olist	Destino en Arquitectura (Ecommify)	Decisión de Diseño
olist_customers_dataset	PostgreSQL: customers	Persistencia relacional para transacciones.
olist_sellers_dataset	PostgreSQL: sellers	Entidad conectada al origen del despacho.
olist_geolocation_dataset	PostgreSQL: customers, sellers	Transformación: Inyectados como tipos nativos GEOMETRY usando PostGIS.
olist_orders_dataset	PostgreSQL: orders	Núcleo transaccional con particionamiento RANGE.
olist_order_items_dataset	PostgreSQL: order_items	Relación 3FN entre pedidos, vendedores y catálogo.
olist_order_payments_dataset	PostgreSQL: payments	Centraliza cobros con JSONB para metadatos.
olist_products_dataset	MongoDB: products	Esquema flexible para atributos variables.
product_category_name...	MongoDB: products	Transformación: Embebidas en el documento mediante <i>Attribute Pattern</i> .
olist_order_reviews_dataset	MongoDB: reviews, products	Transformación: <i>Subset Pattern</i> para anidar reseñas recientes en los productos.

Diseño lógico preliminar- modulo MongoDB

Para el catálogo de productos y el comportamiento analítico, se optó por MongoDB. Su esquema dinámico brinda la flexibilidad necesaria para manejar atributos que varían enormemente según la categoría.

Colecciones principales identificadas y Patrones de Modelado

1. **products:** Almacena el catálogo. Se aplicó el **Patrón de Atributos (Attribute Pattern)** en las especificaciones técnicas (pares clave-valor) para evitar esquemas

caóticos e indexar eficientemente. También se aplicó el **Patrón de Subconjunto (Subset Pattern)** para anidar las reseñas más recientes, ahorrando consultas adicionales desde el Front-End.

2. **reviews:** Centraliza el histórico completo de calificaciones.
3. **user_sessions:** Registra el comportamiento analítico (clics, vistas).

Esquema de documentos (Ejemplo en JSON)

Este es el diseño de un documento para la colección products. Se decidió **anidar** la información de categorías y atributos específicos para evitar los costosos JOINS que tendríamos en SQL.

```
{
  "_id": "prod_8a2b5c",
  "name": "Yamaha FZ25 Custom Parts",
  "brand": "Yamaha",
  "category": {
    "main": "Automotive",
    "sub": "Motorcycle Accessories"
  },
  "description": "Premium custom parts with realistic finish.",
  "photos": [
    "url_image_1.jpg",
    "url_image_2.jpg"
  ],
  "technical_specifications": [
    {"k": "material", "v": "Carbon Fiber"},
  ]
}
```

```
{
  "k": "color", "v": "Matte Black"
},
"metrics": {
  "average_rating": 4.8,
  "total_reviews": 124
},
"recent_reviews": [
  {
    "review_id": "rev_001",
    "rating": 5,
    "comment": "Perfect fit, exact color match."
  }
],
"updated_at": "2026-05-25T17:37:45Z"
}
```

Patrones de modelado a aplicar en MongoDB

Para garantizar la flexibilidad y el rendimiento del catálogo en Ecommify, **hemos decidido implementar** dos patrones de diseño de documentos fundamentales:

- **Patrón de Atributos (Attribute Pattern)**
 - **Dónde se aplicó:** En la colección products, específicamente para el campo technical_specifications.
 - **Justificación:** En un e-commerce, los productos tienen atributos que cambian drásticamente según su categoría (ej. un casco tiene talla, un repuesto tiene material). Para evitar un esquema rígido lleno de valores nulos o un modelo relacional lento, decidimos estructurar estas

especificaciones como un arreglo de pares clave-valor (k y v). Esto permite crear un único índice compuesto y buscar eficientemente cualquier atributo sin importar el tipo de producto.

- **Patrón de Subconjunto (Subset Pattern)**

- **Dónde se aplicó:** En la colección products, creando el campo embebido recent_reviews.
- **Justificación:** Un producto popular puede acumular miles de reseñas, las cuales almaceno en su totalidad en una colección separada (reviews). Sin embargo, para optimizar la experiencia de usuario, he diseñado el documento del producto para que incruste únicamente las 5 reseñas más recientes. Esta decisión arquitectónica me permite renderizar la vista principal del producto en el Front-End con una sola consulta a la base de datos, reduciendo drásticamente la latencia y el consumo de ancho de banda.

Justificación y Matriz de Decisiones Arquitectónicas (Teorema CAP / ACID vs BASE)

Se ha diseñado la siguiente matriz para justificar rigurosamente la separación de cargas de trabajo (híbrido OLTP/OLAP), basándonos en los modelos de consistencia y el Teorema CAP.

Datos / Módulo	Motor	Volumen y Estructura PDF	Consistencia (ACID vs BASE) PDF	Enfoque Teorema CAP PDF+ 1
Pedidos, Pagos, Inventario	PostgreSQL	Medio/Alto. Rígida, normalizada (3FN).	ACID: Requiere atomicidad estricta. Si un pago falla, el inventario no se descuenta.	CA (Consistency + Availability): Prioricé la consistencia absoluta de los datos financieros frente a la tolerancia a particiones.
Catálogo de Productos	MongoDB	Alto. Dinámica y	BASE: Se beneficia de <i>Soft-state</i> . Tolera	AP (Availability + Partition Tolerance): Prioricé la alta

		polimórfica (JSON).	<i>Eventual Consistency.</i>	disponibilidad para que el catálogo siempre responda, distribuyendo la carga.
Reseñas y Analítica	MongoDB	Muy Alto. Basada en documentos anidados.	BASE: No hay riesgo crítico si un dato tarda en propagarse.	AP: Escalamiento horizontal masivo para soportar picos de tráfico analítico.

Análisis de Compensaciones (Trade-offs): Al tomar esta decisión, se asume el trade-off de no poder hacer un JOIN directo entre orders y products. Para mitigar esto, decidimos almacenar los datos inmutables al momento de la compra (como el price) directamente en order_items (PostgreSQL). Así, se mantiene la integridad financiera sin depender de MongoDB para facturación histórica.

Decisiones arquitectónicas justificadas

Matriz de decisión por entidad

Entidad / módulo	Motor recomendado	Justificación
Clientes	PostgreSQL	Requiere integridad, claves únicas, relaciones con pedidos, direcciones y pagos.
Vendedores	PostgreSQL	Maneja datos transaccionales y relaciones con productos, órdenes e inventario.
Pedidos	PostgreSQL	Necesita consistencia ACID, trazabilidad y control transaccional.
Pagos	PostgreSQL	Requiere alta consistencia, auditoría y validaciones estrictas.
Inventario	PostgreSQL	Debe evitar inconsistencias en stock durante compras concurrentes.

Productos	PostgreSQL + MongoDB	PostgreSQL guarda datos principales; MongoDB almacena detalles flexibles como características, imágenes o atributos variables.
Reviews	MongoDB	Son datos semiestructurados, con alto volumen y consultas flexibles.
Catálogo extendido	MongoDB	Permite manejar atributos dinámicos por categoría sin alterar el esquema relacional.
Analítica / dashboards	PostgreSQL con vistas materializadas	Facilita reportes periódicos sobre ventas, clientes y categorías.

Análisis del Teorema CAP aplicado

Para Ecommify se prioriza **consistencia y disponibilidad** en el módulo transaccional. En pedidos, pagos e inventario, PostgreSQL es la mejor opción porque garantiza operaciones ACID y evita errores como pagos duplicados, pedidos incompletos o stock negativo.

MongoDB se usa para información menos crítica, como catálogo extendido y reseñas, donde se acepta mayor flexibilidad y eventual consistencia. En estos casos, la disponibilidad y escalabilidad son más importantes que la consistencia estricta inmediata.

Trade-offs y mitigaciones

Trade-off	Riesgo	Mitigación
Usar PostgreSQL y MongoDB aumenta la complejidad	Duplicidad o desincronización de datos	Definir flujos claros de sincronización y eventos de actualización
MongoDB permite esquemas flexibles	Datos inconsistentes entre documentos	Validaciones a nivel de aplicación y JSON Schema
PostgreSQL garantiza consistencia	Puede crecer el costo de consultas analíticas	Usar particionamiento, índices y vistas materializadas
Catálogo dividido entre SQL y NoSQL	Consultas más complejas	Mantener en PostgreSQL el identificador principal del producto

Anexos

1. Diccionario de datos resumido

Tabla	Campo	Tipo sugerido	Descripción
geolocation	geolocation_zip_code_prefix	INTEGER	Prefijo postal de ubicación
geolocation	geolocation_lat	NUMERIC(10,7)	Latitud
geolocation	geolocation_lng	NUMERIC(10,7)	Longitud
geolocation	geolocation_city	VARCHAR(100)	Ciudad
geolocation	geolocation_state	CHAR(2)	Estado
customers	customer_id	VARCHAR(50)	Identificador del cliente
customers	customer_unique_id	VARCHAR(50)	Identificador único del cliente
customers	customer_zip_code_prefix	INTEGER	Prefijo postal del cliente
customers	customer_city	VARCHAR(100)	Ciudad del cliente
customers	customer_state	CHAR(2)	Estado del cliente
sellers	seller_id	VARCHAR(50)	Identificador del vendedor
sellers	seller_zip_code_prefix	INTEGER	Prefijo postal del vendedor
sellers	seller_city	VARCHAR(100)	Ciudad del vendedor
sellers	seller_state	CHAR(2)	Estado del vendedor
products	product_id	VARCHAR(50)	Identificador del producto

products	product_category_name	VARCHAR(100)	Categoría del producto
products	product_name_length	INTEGER	Longitud del nombre
products	product_description_length	INTEGER	Longitud de la descripción
products	product_photos_qty	INTEGER	Cantidad de fotos
products	product_weight_g	INTEGER	Peso en gramos
products	product_length_cm	INTEGER	Largo en centímetros
products	product_height_cm	INTEGER	Alto en centímetros
products	product_width_cm	INTEGER	Ancho en centímetros
product_category_name_translation	product_category_name	VARCHAR(100)	Nombre original de la categoría
product_category_name_translation	product_category_name_english	VARCHAR(100)	Traducción al inglés
orders	order_id	VARCHAR(50)	Identificador del pedido
orders	customer_id	VARCHAR(50)	Cliente asociado al pedido
orders	order_status	VARCHAR(50)	Estado del pedido
orders	order_purchase_timestamp	TIMESTAMP	Fecha de compra
orders	order_approved_at	TIMESTAMP	Fecha de aprobación
orders	order_delivered_carrier_date	TIMESTAMP	Fecha de entrega al transportador

orders	order_delivered_customer_date	TIMESTAMP	Fecha de entrega al cliente
orders	order_estimated_delivery_date	TIMESTAMP	Fecha estimada de entrega
order_items	order_id	VARCHAR(50)	Pedido asociado
order_items	order_item_id	INTEGER	Secuencia del ítem dentro del pedido
order_items	product_id	VARCHAR(50)	Producto comprado
order_items	seller_id	VARCHAR(50)	Vendedor del producto
order_items	shipping_limit_date	TIMESTAMP	Fecha límite de envío
order_items	price	NUMERIC(12,2)	Precio del producto
order_items	freight_value	NUMERIC(12,2)	Valor del flete
order_payments	order_id	VARCHAR(50)	Pedido asociado al pago
order_payments	payment_sequential	INTEGER	Secuencia del pago
order_payments	payment_type	VARCHAR(50)	Tipo de pago
order_payments	payment_installments	INTEGER	Número de cuotas
order_payments	payment_value	NUMERIC(12,2)	Valor pagado
order_reviews	review_id	VARCHAR(50)	Identificador de la reseña
order_reviews	order_id	VARCHAR(50)	Pedido evaluado
order_reviews	review_score	INTEGER	Calificación

order_reviews	review_comment_title	TEXT	Título del comentario
order_reviews	review_comment_message	TEXT	Mensaje de la reseña
order_reviews	review_creation_date	TIMESTAMP	Fecha de creación
order_reviews	review_answer_timestamp	TIMESTAMP	Fecha de respuesta

2. Scripts SQL preliminares

CREATE TABLE geolocation (

geolocation_zip_code_prefix INTEGER,

geolocation_lat NUMERIC(10,7),

geolocation_lng NUMERIC(10,7),

geolocation_city VARCHAR(100),

geolocation_state CHAR(2)

);

CREATE TABLE customers (

customer_id VARCHAR(50) PRIMARY KEY,

customer_unique_id VARCHAR(50) NOT NULL,

customer_zip_code_prefix INTEGER,

customer_city VARCHAR(100),

customer_state CHAR(2)

);

CREATE TABLE sellers (

seller_id VARCHAR(50) PRIMARY KEY,

seller_zip_code_prefix INTEGER,

```
seller_city VARCHAR(100),  
seller_state CHAR(2)  
);
```

```
CREATE TABLE product_category_name_translation (  
    product_category_name VARCHAR(100) PRIMARY KEY,  
    product_category_name_english VARCHAR(100)  
);
```

```
CREATE TABLE products (  
    product_id VARCHAR(50) PRIMARY KEY,  
    product_category_name VARCHAR(100),  
    product_name_length INTEGER,  
    product_description_length INTEGER,  
    product_photos_qty INTEGER,  
    product_weight_g INTEGER,  
    product_length_cm INTEGER,  
    product_height_cm INTEGER,  
    product_width_cm INTEGER,  
    CONSTRAINT fk_product_category  
        FOREIGN KEY (product_category_name)  
        REFERENCES product_category_name_translation(product_category_name)  
);
```

```
CREATE TABLE orders (  
    order_id VARCHAR(50) PRIMARY KEY,
```

```
customer_id VARCHAR(50) NOT NULL,  
order_status VARCHAR(50),  
order_purchase_timestamp TIMESTAMP,  
order_approved_at TIMESTAMP,  
order_delivered_carrier_date TIMESTAMP,  
order_delivered_customer_date TIMESTAMP,  
order_estimated_delivery_date TIMESTAMP,  
CONSTRAINT fk_orders_customer  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)  
);
```

```
CREATE TABLE order_items (  
    order_id VARCHAR(50) NOT NULL,  
    order_item_id INTEGER NOT NULL,  
    product_id VARCHAR(50) NOT NULL,  
    seller_id VARCHAR(50) NOT NULL,  
    shipping_limit_date TIMESTAMP,  
    price NUMERIC(12,2),  
    freight_value NUMERIC(12,2),  
    PRIMARY KEY (order_id, order_item_id),  
    CONSTRAINT fk_order_items_order  
        FOREIGN KEY (order_id)  
        REFERENCES orders(order_id),  
    CONSTRAINT fk_order_items_product
```

```
FOREIGN KEY (product_id)
REFERENCES products(product_id),
CONSTRAINT fk_order_items_seller
FOREIGN KEY (seller_id)
REFERENCES sellers(seller_id)
);
```

```
CREATE TABLE order_payments (
order_id VARCHAR(50) NOT NULL,
payment_sequential INTEGER NOT NULL,
payment_type VARCHAR(50),
payment_installments INTEGER,
payment_value NUMERIC(12,2),
PRIMARY KEY (order_id, payment_sequential),
CONSTRAINT fk_order_payments_order
FOREIGN KEY (order_id)
REFERENCES orders(order_id)
);
```

```
CREATE TABLE order_reviews (
review_id VARCHAR(50) PRIMARY KEY,
order_id VARCHAR(50) NOT NULL,
review_score INTEGER CHECK (review_score BETWEEN 1 AND 5),
review_comment_title TEXT,
review_comment_message TEXT,
review_creation_date TIMESTAMP,
```

```
review_answer_timestamp TIMESTAMP,  
CONSTRAINT fk_order_reviews_order  
FOREIGN KEY (order_id)  
REFERENCES orders(order_id)  
);
```

3. Índices preliminares

```
CREATE INDEX idx_customers_zip_code  
ON customers(customer_zip_code_prefix);
```

```
CREATE INDEX idx_sellers_zip_code  
ON sellers(seller_zip_code_prefix);
```

```
CREATE INDEX idx_orders_customer_id  
ON orders(customer_id);
```

```
CREATE INDEX idx_orders_purchase_timestamp  
ON orders(order_purchase_timestamp);
```

```
CREATE INDEX idx_order_items_product_id  
ON order_items(product_id);
```

```
CREATE INDEX idx_order_items_seller_id  
ON order_items(seller_id);
```

```
CREATE INDEX idx_order_payments_payment_type
```

```
ON order_payments(payment_type);
```

```
CREATE INDEX idx_order_reviews_order_id
```

```
ON order_reviews(order_id);
```

4. Ejemplos de datos

```
INSERT INTO customers (
```

```
    customer_id,
```

```
    customer_unique_id,
```

```
    customer_zip_code_prefix,
```

```
    customer_city,
```

```
    customer_state
```

```
) VALUES (
```

```
    'CUST001',
```

```
    'UNIQUE001',
```

```
    11001,
```

```
    'bogota',
```

```
    'DC'
```

```
);
```

```
INSERT INTO sellers (
```

```
    seller_id,
```

```
    seller_zip_code_prefix,
```

```
    seller_city,
```

```
    seller_state
```

```
) VALUES (
```

```
    'SELL001',
```

```
50001,  
'medellin',  
'AN'  
);
```

```
INSERT INTO product_category_name_translation (  
    product_category_name,  
    product_category_name_english  
) VALUES (  
    'informatica_acessorios',  
    'computers_accessories'  
);
```

```
INSERT INTO products (  
    product_id,  
    product_category_name,  
    product_name_length,  
    product_description_length,  
    product_photos_qty,  
    product_weight_g,  
    product_length_cm,  
    product_height_cm,  
    product_width_cm  
) VALUES (  
    'PROD001',  
    'informatica_acessorios',
```

```
45,  
320,  
3,  
850,  
30,  
15,  
20  
);
```

```
INSERT INTO orders (  
    order_id,  
    customer_id,  
    order_status,  
    order_purchase_timestamp  
) VALUES (  
    'ORDER001',  
    'CUST001',  
    'delivered',  
    '2025-05-01 10:30:00'  
);
```

```
INSERT INTO order_items (  
    order_id,  
    order_item_id,  
    product_id,  
    seller_id,
```

```
shipping_limit_date,  
price,  
freight_value  
) VALUES (  
  'ORDER001',  
  1,  
  'PROD001',  
  'SELL001',  
  '2025-05-05 23:59:59',  
  120000.00,  
  15000.00  
);
```

```
INSERT INTO order_payments (  
  order_id,  
  payment_sequential,  
  payment_type,  
  payment_installments,  
  payment_value  
) VALUES (  
  'ORDER001',  
  1,  
  'credit_card',  
  3,  
  135000.00  
);
```

```
INSERT INTO order_reviews (  
    review_id,  
    order_id,  
    review_score,  
    review_comment_title,  
    review_comment_message,  
    review_creation_date  
) VALUES (  
    'REV001',  
    'ORDER001',  
    5,  
    'Buen producto',  
    'El producto llegó en buen estado y dentro del tiempo estimado.',  
    '2025-05-10 09:00:00'  
);
```

5. Ejemplo de documento MongoDB complementario

```
{  
    "product_id": "PROD001",  
    "product_category_name": "informatica_acessorios",  
    "category_english": "computers_accessories",  
    "extended_attributes": {  
        "brand": "Generic",  
        "color": "black",  
        "warranty_months": 12,
```

```
"connectivity": ["usb", "bluetooth"]
},
"media": {
  "photos_qty": 3,
  "images": [
    "prod001_front.jpg",
    "prod001_side.jpg",
    "prod001_box.jpg"
  ]
},
"review_summary": {
  "average_score": 5,
  "total_reviews": 1
}
}
```

6. Observación técnica

El anexo se alinea con el diagrama ER propuesto para el módulo transaccional. PostgreSQL se utiliza para las entidades con relaciones fuertes y necesidad de consistencia, como clientes, pedidos, pagos, vendedores, productos e ítems de pedido. MongoDB queda como complemento para información flexible del catálogo, atributos extendidos y datos semiestructurados que no requieren el mismo nivel de normalización.